</> JAYRAIDXYZ · BUILDING IN PUBLIC

Blockchain Dev and Memecoin Fundamentals

Lecture notes: beginner to intermediate

Study guide, lab manual and content bank.
Testnet-first. Educational only. Not financial advice.

5 parts | 27 chapters | Labs and code | 30-day content plan

Version 1.0 · November 2026 · Check official docs: tools and commands change.

Contents

Contents 2

How to Use These Notes 4

Part 1: Foundations 5

1. What Is a Blockchain? 6
2. Wallets, Keys and Staying Safe 8
3. Transactions, Fees and Accounts 10
4. Tokens: Supply, Decimals and Authorities 12
5. Testnets and Devnets: Your Free Lab 14

Part 2: Memecoin Mechanics 16

6. What Is a Memecoin? 17
7. Liquidity Pools and AMMs 19
8. Bonding Curves and Launchpads 21
9. Supply, Holders and On-Chain Signals 23
10. How Rugs Work and How to Spot Them 25
11. Reading On-Chain Data and Tools 27
12. Risk Management and Trader Psychology 28

Part 3: Hands-On Development 30

13. Your Developer Setup 31
14. Create Your First Token on Devnet 33
15. Reading Chain Data with JavaScript 34
16. Your First EVM Token on Sepolia 36
17. Flagship Project: Token Due-Diligence Dashboard 38
18. More Build Projects 40

Part 4: Intermediate: Security, Markets and Automation 41

19. Smart Contract Security Basics 42
20. MEV, Sandwiches and Priority Fees 44
21. Real-Time Data: Subscriptions and Watchers 45
22. Bots and Automation, Safely 46
23. Solana Programs with Anchor 48

Part 5: Content, Freelance and Ethics 50

24. The Content Engine 51
25. 30-Day Content Calendar 53
26. Turning Skills into Freelance Income 55
27. Ethics, Compliance and Reputation 57

Appendices 58

A. Glossary 58
B. Tools and Resources 60
C. 12-Week Roadmap 60
D. Quiz Answer Key 61

How to Use These Notes

These notes take you from zero to confident intermediate level in blockchain development and memecoin market mechanics, using test networks only. Each chapter has goals, a lesson, a lab or code you can run, a short quiz and three content hooks, so studying and posting are the same work.

How these notes are organized

- **Part 1, Foundations**: blockchains, wallets, transactions, tokens and test networks (beginner).
- **Part 2, Memecoin Mechanics**: liquidity, bonding curves, holder analysis, rugs and risk (beginner to lower-intermediate).
- **Part 3, Hands-On Development**: setup, tokens on Solana and Ethereum, reading data in JavaScript, and a flagship project (lower-intermediate).
- **Part 4, Intermediate**: security, MEV, real-time data, bots and Anchor programs.
- **Part 5, Content and Business**: your content engine, a 30-day calendar, freelance offers and ethics.

The learning loop

1. **Read** the lesson and its goals.
2. **Do** the lab on devnet or Sepolia. Type the code yourself.
3. **Check** yourself with the quiz (answers are in Appendix D).
4. **Post** one of the content hooks at the end of the chapter.

Teaching is the study method. You understand a topic far better after explaining it, and your audience watches you learn, which builds trust.

A realistic daily rhythm

Block	Time	What
Learn	60 to 90 minutes	One lesson or lab from the notes
Build	30 to 60 minutes	Progress on your flagship project
Create	30 minutes	Turn the day's learning into a slide set, clip or thread

Ground rules

- Test networks only for anything that involves running transactions.
- No profit promises, no calls, no undisclosed promotion.
- Credit and cite sources, and correct yourself publicly when you are wrong.
- Tools, prices, commands and platform rules change often. Treat the commands and tool names in these notes as a starting point and check the official documentation for current versions before you publish.

PART 1

Foundations

Everything else in these notes sits on five ideas: what a blockchain is, how wallets work, how transactions and fees work, what a token really is, and where you can practise for free. Learn these well and the rest will feel easy.

CHAPTER 1

What Is a Blockchain?

LESSON GOALS

- Explain a blockchain to a non-technical friend in one minute
- Understand hashes, blocks and chains
- Know why consensus matters
- Build and break a tiny blockchain in JavaScript

The one-minute explanation

A blockchain is a shared record book (a **ledger**) that thousands of computers keep identical copies of. Nobody owns it. To add a page, the network must agree the page is valid, and once it is added it is extremely hard to change.

> **NOTE**
> Analogy for your audience: a shared spreadsheet where you can only add new rows, every row carries a fingerprint of the row above it, and thousands of people hold a copy, so nobody can quietly edit history.

Hashes: digital fingerprints

A **hash function** turns any input into a fixed-length string of characters. Change one character of the input and the output changes completely. Blockchains use hashes to link blocks together.

```
import { createHash } from "node:crypto";
const sha = (s) => createHash("sha256").update(s).digest("hex");

console.log(sha("hello"));
console.log(sha("hellp")); // one letter changed, totally different hash
```

Blocks and chains

A **block** holds a batch of transactions, a timestamp and the hash of the previous block. Because every block contains the previous block's hash, editing an old block changes its hash, which breaks every block after it. That chain of hashes is what makes history tamper-evident.

Consensus: how strangers agree

- **Proof of Work** (Bitcoin): miners compete to solve a puzzle. Rewriting history would mean out-working the whole network.
- **Proof of Stake** (Ethereum, Solana): validators lock up (stake) tokens and earn rewards for honest work; cheating can cost them their stake. Solana adds **Proof of History**, a cryptographic clock that helps order transactions quickly.

Term	Meaning
Node	A computer that stores the chain and checks new blocks
Validator	A node that takes part in producing or confirming blocks, usually by staking

Term	Meaning
Finality	The point where a transaction is considered permanent
Layer 1 (L1)	A base chain such as Ethereum or Solana
Layer 2 (L2)	A network built on top of an L1 to be cheaper or faster

Try it: build a mini blockchain

Save this as `chain.mjs` and run `node chain.mjs` (Node 18 or newer).

```
import { createHash } from "node:crypto";
const sha = (s) => createHash("sha256").update(s).digest("hex");
const hashOf = (b) => sha(b.index + JSON.stringify(b.data) + b.prevHash);

class Block {
  constructor(index, data, prevHash) {
    this.index = index;
    this.data = data;
    this.prevHash = prevHash;
    this.hash = hashOf(this);
  }
}

const chain = [new Block(0, "genesis", "0")];
function add(data) {
  const prev = chain[chain.length - 1];
  chain.push(new Block(prev.index + 1, data, prev.hash));
}

const isValid = () =>
  chain.every((b, i) => b.hash === hashOf(b) &&
    (i === 0 || b.prevHash === chain[i - 1].hash));

add({ from: "A", to: "B", amount: 5 });
add({ from: "B", to: "C", amount: 2 });
console.log("valid before tampering:", isValid()); // true

chain[1].data.amount = 500; // rewrite history
console.log("valid after tampering:", isValid());  // false
```

The second check fails because block 1's stored hash no longer matches its contents. Real blockchains add consensus on top so that nobody can simply recompute the hashes and get away with it.

Check yourself

1. What happens to later blocks if you edit block 1?
2. Name the two main consensus models and one chain for each.
3. What is the difference between a node and a validator?

CONTENT HOOKS: POST ONE OF THESE

→ Explain a blockchain with the shared-spreadsheet analogy (4 slides)

→ I broke my own blockchain on purpose (screen recording of the tamper demo)

→ Change one letter, get a totally different hash (10-second clip)

CHAPTER 2

Wallets, Keys and Staying Safe

LESSON GOALS

- Know the difference between private key, public key and seed phrase
- Understand custodial and non-custodial wallets
- Follow a basic security routine
- Set up a dedicated development wallet

The three things to understand

- **Private key**: the secret that proves you control an address. Anyone who has it controls the funds.
- **Public key / address**: derived from the private key. Safe to share. It is how people send you assets.
- **Seed phrase** (recovery phrase): 12 or 24 words that can regenerate all your keys. Treat it as the master key to everything.

A wallet app does not hold coins. Coins live on the chain. The wallet holds your keys and lets you **sign** transactions.

Custodial vs non-custodial

On an exchange the company holds the keys (custodial). In Phantom, Solflare or MetaMask you hold the keys (non-custodial). That means freedom, but also that nobody can reset a lost seed phrase for you.

Wallet security rules

- Never type your seed phrase into a website, DM, form or support chat. Real support never asks for it.
- Use a separate wallet for development and testing. Your devnet wallet should never hold real funds.
- Treat every signature request as permission. Read what you sign and reject anything unexpected.
- On EVM chains, review token approvals and revoke old ones.
- Bookmark real sites. Ignore links from strangers and airdrop DMs.
- Back up your seed phrase offline (paper or metal), never as a screenshot or in cloud notes.
- Once you hold anything of real value, use a hardware wallet.

CAUTION

Scams aimed at newcomers are everywhere: fake support accounts, fake airdrops, fake wallet-validation pages and malicious browser extensions. Assume any unsolicited message is a scam until proven otherwise.

Try it: set up your lab

1. Create a new browser profile called **dev**.
2. Install Phantom (Solana) and MetaMask (EVM) in that profile only.
3. Create brand-new wallets and write the seed phrases on paper.
4. Switch each wallet to its test network (Solana devnet, Ethereum Sepolia).

Never import a wallet that you use for real money into this lab.

Check yourself

1. Which of these is safe to share: private key, public address, seed phrase?
2. Why use a separate wallet for development?
3. What does signing a transaction actually do?

> **CONTENT HOOKS: POST ONE OF THESE**
>
> → 5 wallet security rules I follow as a dev
>
> → How wallet drainers trick people (and the 3 signs to spot them)
>
> → My dev lab: a separate browser profile and test wallets

CHAPTER 3

Transactions, Fees and Accounts

LESSON GOALS

- Read the parts of a transaction
- Understand fees on Ethereum and Solana
- Compare the two account models
- Read a transaction on a block explorer

Anatomy of a transaction

- **Signer(s)**: who authorized it.
- **Instructions**: what to do (transfer, swap, mint, approve).
- **Fee**: what the sender pays the network.
- **Result**: success or failure, plus logs.
- **Position**: the block or slot and timestamp.

A transaction is a signed message. Nodes check the signature, run the instructions and record the result.

Fees

- **Ethereum**: gas used times gas price, paid in ETH. 1 ETH = 10^18 wei.
- **Solana**: a small base fee per signature (5,000 lamports) plus optional priority fees. 1 SOL = 1,000,000,000 lamports.
- Failed transactions can still cost a fee.

Two account models

Ethereum has externally owned accounts (wallets) and contract accounts, where code and storage live together. Solana separates them: **programs** are stateless executable accounts, and data lives in separate accounts that programs own. Solana accounts must hold a minimum balance (rent exemption) to stay on chain.

Topic	Ethereum (EVM)	Solana
Contract language	Solidity	Rust, often with Anchor
Token standard	ERC-20	SPL Token (Token Program)
Fee currency	ETH	SOL
Dev test network	Sepolia	Devnet
Common wallets	MetaMask	Phantom, Solflare
Explorers	Etherscan	Solscan, Solana Explorer
Memecoin scene	Active on Base, Ethereum, BNB Chain	Very active, cheap and fast launches

Reading a transaction on an explorer

1. Paste the signature or hash into the explorer search.
2. Check the status (success or failed) and the fee.
3. Find who signed it and which accounts it touched.
4. Read the instruction list and the logs to see what actually happened.

Check yourself

1. How many lamports are in 1 SOL?
2. Where does program state live on Solana compared with Ethereum?
3. Can a failed transaction cost a fee?

CONTENT HOOKS: POST ONE OF THESE

→ Ethereum vs Solana for developers in 4 slides
→ How to read a transaction on Solscan (screen recording)
→ Why Solana fees are tiny and what priority fees are

CHAPTER 4

Tokens: Supply, Decimals and Authorities

LESSON GOALS

- Tell native coins from tokens
- Understand ERC-20 and SPL tokens
- Work with decimals and supply
- Know why mint and freeze authorities matter

A **native coin** (ETH, SOL) pays fees and secures the network. A **token** is created by a program or contract running on the chain. Creating one is cheap and open to anyone, which is exactly why there are so many memecoins.

ERC-20 on Ethereum-style chains

An ERC-20 token is a smart contract that keeps a table of balances plus functions such as `transfer` and `approve`. The contract is the token.

SPL tokens on Solana

- **Mint account**: defines the token (supply, decimals, authorities).
- **Token account**: holds one wallet's balance of one token. The associated token account (ATA) is the standard one per wallet per mint.
- **Metadata**: name, symbol and image are stored separately, commonly with Metaplex Token Metadata.
- The **mint address** is the token's true identity.

CAUTION

Names and tickers are not unique. Anyone can create another token with the same name and logo. Always verify the mint or contract address from an official source.

Decimals and supply

Blockchains store whole numbers. A token with 6 decimals and a raw amount of 1,000,000 is displayed as 1.0.

```
// raw amount = display amount * 10^decimals
const decimals = 6;
const display = 250_000;
const raw = BigInt(display) * 10n ** BigInt(decimals);
console.log(raw); // 250000000000n
```

Authorities on Solana

- **Mint authority**: can create more tokens. If it is still active, supply can inflate at will.
- **Freeze authority**: can freeze token accounts so holders cannot transfer.
- Setting an authority to none (null) revokes it permanently.

These two fields are among the first things to check on any token. Revoked authorities remove some risks but do not make a token safe.

Burning destroys tokens and lowers the total supply.

Check yourself

1. What identifies a token for certain: its name, ticker or mint address?
2. A token has 9 decimals and raw balance 5,000,000,000. What is the display balance?
3. What risk does an active mint authority create?

CONTENT HOOKS: POST ONE OF THESE

→ What is a mint authority? (and why you should check it)
→ Same name, different token: how impersonation works
→ Decimals explained in 3 slides

CHAPTER 5

Testnets and Devnets: Your Free Lab

LESSON GOALS

- Know which network to use for what
- Get free test coins
- Avoid common testnet mistakes

A test network behaves like the real chain but its coins have no value. Developers use them to build, test and break things safely. Every lesson in these notes that involves running code uses a test network.

Network	Chain	Purpose
Devnet	Solana	Developer testing, free airdropped SOL
Testnet	Solana	Mostly for validator and core testing
Mainnet-beta	Solana	The real network with real money
Sepolia	Ethereum	Developer testing for smart contracts

Getting free test coins

- **Solana devnet**: run `solana airdrop 2` or use the web faucet. Requests are rate limited, so retry later.
- **Sepolia**: free faucets are run by infrastructure providers. Some require a login or a small mainnet balance. Search for a Sepolia faucet and use well-known providers.

Rules for working on test networks

- Test coins have no value. Never buy or sell them, and ignore anyone who offers to.
- Devnet can be reset or rate limited. Do not rely on it for anything permanent.
- Use an RPC provider (Helius, Alchemy, QuickNode and others) for stable endpoints. Public endpoints are rate limited.
- Check which network your wallet and CLI are on before you sign anything.

NOTE

Content habit: put the network name (Devnet or Sepolia) on every screenshot and slide. It keeps your audience clear that nothing you show is a real-money trade.

Check yourself

1. Is Solana devnet or testnet the one for general app development?
2. Why should you never trade test tokens for money?
3. Name two ways to get test coins.

CONTENT HOOKS: POST ONE OF THESE

→ Free money? Devnet explained: why test coins are worthless
→ My first devnet airdrop (screen recording)
→ Devnet vs testnet vs mainnet in one slide

PART 2

Memecoin Mechanics

Memecoins are a fast, public, fully transparent case study of how tokens, liquidity and markets behave. Treat this part as market anatomy: learn how the machinery works so that you can explain it, and so that you can spot risk. Nothing here is a trading recommendation.

CHAPTER 6

What Is a Memecoin?

LESSON GOALS

- Define a memecoin and its life cycle
- Understand why price depends on attention and liquidity
- Know why this niche is high risk

A **memecoin** is a token whose value comes mainly from a joke, character, trend or community rather than from a product or cash flow. Its price is driven by attention and by how much liquidity sits behind it. That makes it extremely volatile, and the large majority of memecoins end up with almost no value.

The life cycle

Stage	What happens	What to watch
1. Narrative	A joke, character, news event or trend	Original idea or a copy?
2. Launch	Token is created on a launchpad or manually	Creator wallet and supply split
3. Early buying	Snipers, insiders and bundled wallets buy first	Who bought first, and are the wallets linked?
4. Hype	Social posts, trending lists, paid promotion	Bot-like engagement, undisclosed ads
5. Peak	Thin liquidity compared with market cap	Holder concentration, rising sell pressure
6. Decay	Attention moves on and volume dries up	Where most tokens end

Launchpads

Launchpads let anyone create a token in minutes for a very small fee. pump.fun is the best-known example on Solana. Low cost means huge numbers of launches and very few survivors. Platform rules and features change often, so always read the current documentation before you explain how one works.

Why study them

- All the data is public and on-chain, so you can show your working.
- They teach AMMs, bonding curves, MEV and risk analysis in one place.
- Audience interest is high, and teaching how to spot risk is a credible, useful angle.

CAUTION

Nothing in these notes is financial advice. Most people who trade memecoins lose money. Do not promise returns, do not call tokens, and do not present screenshots of wins as proof of skill.

Check yourself

1. What mainly drives a memecoin's price?
2. Why are there so many memecoins?
3. Why is teaching how to spot risk a strong content angle?

CONTENT HOOKS: POST ONE OF THESE

→ The 6 stages of a memecoin's life in one carousel
→ Why most memecoins die (and what the on-chain data shows)
→ What a launchpad actually does, step by step

CHAPTER 7

Liquidity Pools and AMMs

LESSON GOALS

- Understand how an automated market maker sets prices
- Work through the x * y = k math
- Explain slippage and price impact
- Know why market cap is not exit liquidity

Traditional exchanges match buyers and sellers in an order book. An **automated market maker (AMM)** replaces that with a **liquidity pool**: a smart contract holding two assets, for example SOL and a memecoin. Traders swap against the pool, and a formula sets the price.

The constant-product rule

Many AMMs keep the product of the two reserves constant: **x * y = k**. When you buy tokens, you add SOL and remove tokens, and the price moves along the curve.

Worked example

A pool holds 100 SOL and 1,000,000 TOKEN, so k = 100,000,000. You buy with 10 SOL (fees ignored).

Step	SOL in pool	TOKEN in pool	Spot price (SOL per token)
Before	100	1,000,000	0.000100
After your trade	110	909,090.9	0.000121

You receive 90,909.1 tokens. Your average price was 0.000110, which is 10 percent above the starting price. After your trade the pool's price is 21 percent higher. That gap is **price impact**, and it grows quickly as your trade gets bigger compared with the pool.

```
// Constant-product AMM (x * y = k) with a 0.3% fee
function swapExactIn(reserveIn, reserveOut, amountIn, fee = 0.003) {
  const inAfterFee = amountIn * (1 - fee);
  const amountOut = (reserveOut * inAfterFee) / (reserveIn + inAfterFee);
  const spot = reserveIn / reserveOut;          // price before the trade
  const paid = amountIn / amountOut;            // average price you paid
  return {
    amountOut,
    newReserveIn: reserveIn + amountIn,
    newReserveOut: reserveOut - amountOut,
    priceImpact: paid / spot - 1,
  };
}

// 100 SOL / 1,000,000 TOKEN pool, buy with 10 SOL
console.log(swapExactIn(100, 1_000_000, 10));
// amountOut is about 90,661 and priceImpact is about 0.103
```

Slippage

Slippage is the difference between the price you expected and the price you got, because the pool moved between your click and your transaction landing. Wallets let you set a slippage tolerance. Too high and you can be exploited. Too low and your trade may fail.

> **CAUTION**
> Market cap is not exit liquidity. A token with a 10 million dollar market cap and 20 thousand dollars of liquidity cannot be sold for anything close to 10 million. Selling even a small fraction crashes the price.

Check yourself

1. In x * y = k, what happens to the token price when someone buys?
2. Why does a big trade in a small pool cause large price impact?
3. Why is market cap a poor measure of how much you can sell for?

> **CONTENT HOOKS: POST ONE OF THESE**
> → I buy 10 SOL in a 100 SOL pool: here is the math (3 slides)
> → Slippage explained with one number
> → Why 'market cap' lies about what you can sell

CHAPTER 8

Bonding Curves and Launchpads

LESSON GOALS

- Understand price as a function of supply
- Calculate the cost of buying along a curve
- Explain graduation to a liquidity pool

A **bonding curve** is a rule that sets a token's price from its current supply. The contract itself sells tokens to buyers and buys them back from sellers along the curve, so there is no need for a separate liquidity provider at the start. The more that has been bought, the higher the next token costs.

A simple linear curve

If price = k * supply, the cost of moving supply from s1 to s2 is the area under the line: (k / 2) * (s2^2 - s1^2). Each new batch of tokens costs more than the last.

```
// Toy linear bonding curve: price(s) = k * s  (toy numbers, not a real platform)
const k = 0.000000001;
const cost = (s1, s2) => (k / 2) * (s2 ** 2 - s1 ** 2);

let supply = 0;
function buy(tokens) {
  const c = cost(supply, supply + tokens);
  supply += tokens;
  return c;
}

console.log(buy(100_000)); // about 5 (first 100k tokens)
console.log(buy(100_000)); // about 15 (next 100k cost 3x more)
```

Virtual reserves and graduation

Some launchpads model the curve like a constant-product pool using **virtual reserves**, numbers that exist only in the maths and give a starting price. When enough has been bought (the curve's target), the remaining tokens and the funds raised are moved into a regular liquidity pool. This step is called **graduation** or **migration**.

	Bonding curve	AMM pool
Price rule	Formula based on supply sold	Formula based on reserves (x * y = k)
Where liquidity comes from	The curve's own reserves	Liquidity providers or the migrated funds
Typical stage	Launch phase	After graduation

NOTE

Exact parameters, fees and migration targets differ by platform and change over time. Before you teach a specific platform, read its current documentation and cite it.

Check yourself

1. Why does each batch of tokens cost more along a rising curve?
2. What is graduation?
3. What is a virtual reserve?

CONTENT HOOKS: POST ONE OF THESE

→ How a bonding curve prices tokens (build it in 15 lines)
→ What 'graduating' means on a launchpad
→ Bonding curve vs AMM pool: the differences in one table

CHAPTER 9

Supply, Holders and On-Chain Signals

LESSON GOALS

- Read who holds a token and how that affects risk
- Understand bundles and snipers
- Tell market cap from FDV

Because everything is on-chain, you can inspect a token's distribution. These are the signals analysts look at first.

Signal	What it means	Why it matters
Top-10 holder share	Share of supply held by the largest wallets (excluding pools and program accounts)	High concentration lets a few wallets crash the price
Creator holdings	Share held by the creator and linked wallets	Large holdings can be dumped
Bundled buys	Many wallets buying in the same block, funded from one source	Looks like demand but may be one actor
Snipers	Wallets that buy in the first seconds	Often bots that sell into later buyers
Mint / freeze authority	Revoked or still active	Active means inflation or frozen accounts are possible
Liquidity	Depth of the pool; on EVM, whether LP tokens are locked or burned	Thin or removable liquidity means exit risk
Holder count	Unique wallets holding the token	Can be faked by splitting funds across wallets
Volume vs liquidity	Trading volume compared with pool size	Wash trading can inflate volume

Market cap vs FDV

Market cap = price x circulating supply. **Fully diluted valuation (FDV)** = price x total supply. For many memecoins all supply is already circulating, so they are equal, but you should confirm it. For tokens with locked or unreleased supply, FDV is the more honest number.

CAUTION

No single signal proves a token is safe or a scam. Combine them, check the data yourself, and be honest about what you do not know.

Check yourself

1. What does a bundled buy look like on-chain?
2. When would FDV matter more than market cap?
3. Why can holder count mislead?

CONTENT HOOKS: POST ONE OF THESE

→ 8 on-chain signals I check on any token
→ Bundles explained: how one wallet fakes a crowd
→ I analyzed a token's top holders (screen recording)

CHAPTER 10

How Rugs Work and How to Spot Them

LESSON GOALS

- Recognize the common rug and scam patterns
- Use a due-diligence checklist
- Explain risks without promoting tokens

A **rug pull** is any situation where the people who control a token or its liquidity take value from holders. Learn the patterns at concept level so that you can recognize and explain them.

Pattern	How it works (concept)	What to check
Liquidity pull (EVM)	Creator removes liquidity from the pool, leaving holders unable to sell at a real price	Are LP tokens locked or burned?
Hidden mint / inflation	An owner creates new supply and sells it	Mint authority revoked? Owner-only mint functions?
Honeypot	Contract logic lets you buy but blocks or heavily taxes selling	Verified source, sell tax, a test sell simulation, trusted scanners
Insider dump	Team or linked wallets sell into the hype	Holder distribution, wallet links, bundles
Impersonation	Copy of a known token's name and logo at a different address	Verify the address from official sources
Drainer / phishing	Fake site or approval request that empties a wallet	Do not connect to unknown sites; read signatures
Wash trading	The same actors trade back and forth to fake volume	Volume vs unique traders

NOTE

Many bad outcomes follow a platform's rules and are still painful for buyers. 'Not technically a rug' is not the same as 'safe'.

Pre-trade due diligence checklist

Use this to analyze tokens and as a content template. Run it on devnet tokens you create, or on live tokens purely as a study exercise.

1. Confirm the mint or contract address from an official source.
2. Check mint and freeze authorities (Solana) or owner functions (EVM).
3. Check how liquidity is held: locked, burned, or controlled by one wallet.
4. Look at the top holders and the creator's share.
5. Look for bundled or linked wallets at launch.
6. Compare volume with liquidity and unique traders.
7. Read the verified contract source where available.
8. Check whether sells work (simulate, or read the sell tax).
9. Search for the project's history and the creator's past launches.

10. Ask what you are missing, then write down your reasoning.

Devnet lab: authority risk

In the next part you will create a token on devnet, mint more supply to yourself, then revoke the mint authority and show that minting fails afterwards. It makes a clear, safe demonstration of why authorities matter.

Check yourself

1. What is a honeypot?
2. Why is 'the contract is not a scam' not enough?
3. Name three things on the checklist that you can check without any code.

CONTENT HOOKS: POST ONE OF THESE

→ 7 ways memecoin rugs work (concept only, 7 slides)
→ My 10-step token due-diligence checklist
→ Revoking mint authority on devnet: the demo

CHAPTER 11

Reading On-Chain Data and Tools

LESSON GOALS

- Use explorers and analytics sites
- Walk through a token page step by step
- Understand the limits of automated scores

Tool type	Examples	Use for
Block explorer	Solscan, Solana Explorer, Etherscan	Transactions, accounts, token details, verified contracts
DEX analytics	DexScreener, Birdeye	Charts, pools, liquidity, trades
Risk scanners	RugCheck and similar tools	Quick red-flag scans (never the final word)
RPC providers	Helius, Alchemy, QuickNode	Reading data and sending transactions from code

NOTE

Tools and sites change often. Verify that a tool is current and reputable before you recommend it, and say which one you used in your content.

Walkthrough: reading a token page

1. Copy the mint address and open it on an explorer.
2. Check decimals, supply and the two authorities.
3. Open the holders tab and note the top-10 share.
4. Find the creation transaction and the creator wallet.
5. Open the pool on an analytics site and note liquidity and age.
6. Compare volume, trades and unique wallets.

CAUTION

Automated risk scores can be wrong in both directions. Use them to know where to look, then check the data yourself.

Check yourself

1. Which tool type tells you who holds a token?
2. Why not trust one risk score?
3. Why name the tool you used in your content?

CONTENT HOOKS: POST ONE OF THESE

→ How I read a token on Solscan in 60 seconds
→ Explorer vs analytics site: what each is for
→ Why I never trust a single 'rug score'

CHAPTER 12

Risk Management and Trader Psychology

LESSON GOALS

- Understand why most traders lose
- Practise with a paper-trading journal
- Know your compliance responsibilities

Attention-driven markets punish emotion. Common traps: fear of missing out, chasing green candles, averaging down on a dying token, and only remembering the wins. On-chain analyses repeatedly find that most retail memecoin traders end up with losses.

Principles

- Only ever risk money you can afford to lose completely.
- Size positions small enough that one failure cannot hurt you.
- Decide your reasoning and exit rules before you act, not during.
- Beware survivorship bias: you see the winners' screenshots, not the many who lost.
- Never use leverage or borrowed money for speculation.

Paper trading and a trade journal

Practise with simulated trades and track everything in a spreadsheet. The goal is learning, not profit.

Field	Example
Date and token	2026-11-10, TESTMEME (devnet)
Reason for entry	Why you think this setup matters
Size and price	Simulated amount and entry price
Exit rule	Condition decided before entry
Result	Outcome after exit
Mistake or lesson	What you would change

Responsibilities

- Rules on taxes, promotions and crypto services differ by country. Check your local rules before you give any financial content or earn from it.
- Disclose any sponsorship, referral link or token you hold when you post about it.

CAUTION

This course teaches how markets and code work. It does not tell anyone what to buy or sell, and you should say that clearly in your content.

Check yourself

1. What is survivorship bias in crypto content?
2. What should you decide before you enter a simulated trade?
3. What must you disclose when you post about a token?

CONTENT HOOKS: POST ONE OF THESE

→ The 5 mistakes that wipe out most memecoin traders
→ My paper-trading journal template
→ What survivorship bias hides in crypto Twitter

PART 3

Hands-On Development

From here you write and run code. Every example uses Solana devnet or Ethereum Sepolia, so nothing you do can cost real money. Build the habit of recording what you do: each lab in this part is also a ready-made piece of content.

CHAPTER 13

Your Developer Setup

LESSON GOALS

- Install the core tools
- Configure the Solana CLI for devnet
- Keep keys and secrets out of your code

- **Node.js 20 or newer** and a package manager (npm or pnpm)
- **Git and GitHub** for version control and your public portfolio
- **VS Code** or another editor
- **Rust** (via rustup) and the **Solana CLI** for Solana work
- **Anchor** for Solana programs (covered in the intermediate part)
- **Foundry, Hardhat or the Remix browser IDE** for Solidity
- An **RPC provider** account (Helius, Alchemy, QuickNode or similar)

Solana CLI on devnet

```
# Install the Solana CLI (check the official docs for the current command)
sh -c "$(curl -sSfL https://release.anza.xyz/stable/install)"
solana --version

# Point it at devnet and create a dev-only keypair
solana config set --url devnet
solana-keygen new --outfile ~/.config/solana/devnet.json
solana config set --keypair ~/.config/solana/devnet.json

solana address       # your devnet address
solana airdrop 2     # free test SOL
solana balance
```

CAUTION

That keypair file is a private key. Add it to .gitignore, never commit it, and never reuse it on mainnet.

Project hygiene

- Keep RPC keys and secrets in a `.env` file that is listed in `.gitignore`.
- Never put a private key or seed phrase in code, screenshots or a public repo.
- Call RPC providers from a server route, not from the browser, when the key is not meant to be public.
- Write down the versions you used. Tutorials age quickly.

Check yourself

1. Why add the keypair file to .gitignore?
2. Which command gives you free devnet SOL?
3. Why call a paid RPC from a server route?

CONTENT HOOKS: POST ONE OF THESE

→ My Web3 dev setup in 2026: every tool I use
→ Set up the Solana CLI in 60 seconds (screen recording)
→ The one file you must never commit to GitHub

CHAPTER 14

Create Your First Token on Devnet

LESSON GOALS

- Create, mint and transfer an SPL token
- See authorities in action
- Revoke the mint authority

Install the token CLI with `cargo install spl-token-cli`, make sure you are on devnet, then run the commands below. Replace <MINT> with the address printed by the first command.

```
spl-token create-token --decimals 6      # prints the MINT address
spl-token create-account <MINT>          # your token account for this mint
spl-token mint <MINT> 1000000            # mint 1,000,000 tokens to yourself
spl-token supply <MINT>                  # total supply
spl-token balance <MINT>                 # your balance
spl-token transfer <MINT> 100 <RECIPIENT_ADDRESS> --fund-recipient
```

- **create-token** makes the mint account. You become the mint and freeze authority.
- **create-account** makes your token account (ATA) so you can hold the token.
- **mint** creates new supply into your account.
- **transfer** sends tokens, and `--fund-recipient` creates the recipient's token account if needed.

Lab: why authorities matter

```
spl-token mint <MINT> 500                  # works: you still hold the mint authority
spl-token authorize <MINT> mint --disable  # revoke it permanently
spl-token mint <MINT> 500                  # now fails: nobody can mint again
```

Open the mint on Solana Explorer and add `?cluster=devnet` to the URL. Compare the mint authority field before and after. This one demo shows beginners exactly what to look for on a real token.

NOTE

Name, symbol and image come from Metaplex token metadata, a separate step. It makes a good follow-up lab once you are comfortable with the basics.

Check yourself

1. What does the mint authority control?
2. Can a revoked mint authority be restored?
3. Why does a recipient need a token account?

CONTENT HOOKS: POST ONE OF THESE

→ I created a token in 3 commands (devnet)
→ Before and after: revoking mint authority on the explorer
→ Mint, transfer, burn: the full token lifecycle in one video

CHAPTER 15

Reading Chain Data with JavaScript

LESSON GOALS

- Connect to devnet from Node
- Fetch balances, activity and token holders
- Parse mint details and authorities

```
mkdir chain-reader && cd chain-reader
npm init -y
npm pkg set type=module
npm i @solana/web3.js
```

Balance and recent activity

```
import { Connection, PublicKey, clusterApiUrl, LAMPORTS_PER_SOL }
  from "@solana/web3.js";

const connection = new Connection(clusterApiUrl("devnet"), "confirmed");
const wallet = new PublicKey("PASTE_DEVNET_ADDRESS");

const lamports = await connection.getBalance(wallet);
console.log("SOL:", lamports / LAMPORTS_PER_SOL);

const sigs = await connection.getSignaturesForAddress(wallet, { limit: 5 });
for (const s of sigs) console.log(s.signature, s.err ? "failed" : "ok");
```

Mint details and authorities

```
const mint = new PublicKey("PASTE_MINT_ADDRESS");

const info = await connection.getParsedAccountInfo(mint);
const d = info.value.data.parsed.info;
console.log({
  decimals: d.decimals,
  supply: d.supply,
  mintAuthority: d.mintAuthority,       // null means revoked
  freezeAuthority: d.freezeAuthority,   // null means revoked
});
```

Top holders and concentration

```
const largest = await connection.getTokenLargestAccounts(mint);
const supply = await connection.getTokenSupply(mint);
const total = Number(supply.value.uiAmount);

for (const a of largest.value.slice(0, 10)) {
  const pct = (a.uiAmount / total) * 100;
  console.log(a.address.toBase58(), pct.toFixed(2) + "%");
}
```

NOTE

Library versions move quickly. This uses @solana/web3.js 1.x, which many tutorials and tools still rely on. Newer Solana JavaScript libraries exist, so check the official docs for the current recommendation. The addresses returned are token accounts, so on a real token some belong to pools or programs rather than people.

Check yourself

1. What does mintAuthority equal when it has been revoked?
2. What does getTokenLargestAccounts return?
3. Why is a top token account not always a person?

CONTENT HOOKS: POST ONE OF THESE

→ Check any token's authorities with 10 lines of JavaScript
→ Build a top-holders checker (live coding)
→ How to read a wallet's activity from code

CHAPTER 16

Your First EVM Token on Sepolia

LESSON GOALS

- Write a basic ERC-20 in Solidity
- Deploy it to Sepolia with Remix
- Recognize owner powers when reading a contract

1. Open the Remix browser IDE and create a new file, for example `TestMeme.sol`.
2. Paste the contract below and compile with Solidity 0.8.20 or newer.
3. Switch MetaMask to Sepolia and fund it from a faucet.
4. In Remix choose the injected provider, then deploy.
5. Add the token address to MetaMask and find it on Sepolia Etherscan.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract TestMeme is ERC20 {
    constructor() ERC20("Test Meme", "TMEME") {
        _mint(msg.sender, 1_000_000 * 10 ** decimals());
    }
}
```

OpenZeppelin provides audited building blocks. This version mints the whole supply once at deployment and has no way to create more, which is the simplest and lowest-risk design.

What an owner-controlled token looks like

```
contract RiskyToken is ERC20, Ownable {
    constructor() ERC20("Risky", "RSK") Ownable(msg.sender) {}

    // The owner can create unlimited supply at any time
    function mint(address to, uint256 amount) external onlyOwner {
        _mint(to, amount);
    }
}
```

Neither contract is malicious by itself: owner-controlled minting is a normal pattern. The point is to know what to look for when you read a contract. An owner function that can mint, pause transfers, blacklist wallets or change fees gives someone power over holders. `renounceOwnership()` gives that power up permanently.

Check yourself

1. What does onlyOwner do?
2. What does renouncing ownership do?
3. Why use OpenZeppelin instead of writing ERC-20 from scratch?

CONTENT HOOKS: POST ONE OF THESE

→ Deploy an ERC-20 in 5 minutes with Remix
→ Owner powers in a token contract: what to look for
→ Solana token vs Ethereum token: same idea, different design

CHAPTER 17

Flagship Project: Token Due-Diligence Dashboard

LESSON GOALS

- Plan and ship a portfolio-grade project
- Turn the build into weeks of content

This is your showcase. It uses everything from Parts 1 to 3, fits your Next.js skills, proves you can build Web3 products for clients, and gives you a content series. It is a **read-only analysis tool**: no trading and no calls to buy anything.

What it does

1. The user pastes a token mint address (start on devnet).
2. The server fetches decimals, supply, mint authority and freeze authority.
3. It fetches the top holders and calculates concentration.
4. It lists plain-language flags and a clear notice that it is not advice.

```
// app/api/token/route.js (Next.js)
import { Connection, PublicKey } from "@solana/web3.js";

export async function GET(req) {
  const mintStr = new URL(req.url).searchParams.get("mint");
  const connection = new Connection(process.env.RPC_URL, "confirmed");
  const info = await connection.getParsedAccountInfo(new PublicKey(mintStr));
  const d = info.value.data.parsed.info;

  const flags = [];
  if (d.mintAuthority) flags.push("Mint authority is still active");
  if (d.freezeAuthority) flags.push("Freeze authority is still active");

  return Response.json({ decimals: d.decimals, supply: d.supply, flags });
}
```

Four-week plan

Week	Build	Content
1	Node script for authorities and supply	Live coding clip plus a 'what is an authority' carousel
2	Next.js UI with the API route	Screen recording of the first working version
3	Holders table and concentration chart	Thread on what concentration means
4	Deploy to Vercel and write the case study	Launch post and portfolio case study

NOTE

Add a clear disclaimer in the UI. Open-source the repo and write a short case study. For freelance leads, 'I built this and here is the code' beats any claim.

Check yourself

1. Why keep the RPC call on the server?
2. Why make the tool read-only?
3. What turns a project into a portfolio piece?

CONTENT HOOKS: POST ONE OF THESE

→ I'm building a token checker in public: week 1
→ Build log: my Next.js dashboard reads any Solana token
→ Case study: from idea to deployed Web3 tool

CHAPTER 18

More Build Projects

LESSON GOALS

- Pick the next projects for skills and content

Project	Skills practised	Level	Content angle
Wallet tracker on devnet	RPC, parsing, UI	Beginner	Build log and demo
AMM swap simulator (web)	Maths, charts, React	Beginner	Interactive explainer
Bonding-curve visualizer	Maths, charts	Beginner	How launchpad pricing works
Paper-trading journal app	Forms, storage, charts	Intermediate	Learn risk management in public
'Explain this transaction' tool	Parsing instructions and logs	Intermediate	Teach by decoding real transactions
New-mint watcher (devnet)	Websockets, filters	Intermediate	Real-time dashboards
Anchor counter, then a simple vault	Rust, Anchor, tests	Intermediate	My first Solana program

Ship small, finish things, and write each one up. A short public repo with a clear README and a demo link is worth more than a big unfinished idea.

Check yourself

1. Which project suits a beginner who wants a quick visual win?
2. What should every project end with?

CONTENT HOOKS: POST ONE OF THESE

→ 7 Web3 projects that get you hired

→ Build an AMM simulator with React (time-lapse)

PART 4

Intermediate: Security, Markets and Automation

Now you go one level deeper: how contracts get attacked, how traders get exploited, how to work with live data and how to build tools safely. These chapters separate someone who follows tutorials from someone clients can trust.

CHAPTER 19

Smart Contract Security Basics

LESSON GOALS

- Know the most common vulnerability classes
- Apply the checks-effects-interactions pattern
- Understand Solana-specific account checks

Issue	Plain-language explanation	Defence
Access control	A sensitive function can be called by anyone	Restrict with roles, owner checks and signer checks
Reentrancy	An external call lets an attacker re-enter before state is updated	Update state first; use a reentrancy guard
Unchecked calls	A failed external call is ignored	Check return values and revert on failure
Oracle manipulation	A price feed that can be moved in one transaction	Use robust, time-averaged price sources
Front-running	Others see your pending transaction and act first	Limits on slippage, commit-reveal designs, private routes
Missing signer / owner checks (Solana)	A program accepts accounts without verifying who signed or who owns them	Validate every account; use Anchor constraints

NOTE

Solidity 0.8 and later reverts on integer overflow automatically, which removed a whole class of old bugs. Older contracts and unchecked blocks can still be affected.

Checks-effects-interactions

```
// Withdraw pattern: update state BEFORE sending funds
function withdraw() external {
    uint256 amount = balances[msg.sender];
    require(amount > 0, "nothing to withdraw");   // checks
    balances[msg.sender] = 0;                     // effects
    (bool ok, ) = msg.sender.call{value: amount}("");  // interaction
    require(ok, "transfer failed");
}
```

Self-review checklist

- Who is allowed to call each function, and is that enforced?
- What power does the owner or admin have over users?
- Is state updated before any external call?
- Does every account or input get validated?
- What is the worst thing an attacker could do with one transaction?

NOTE

Writing about vulnerabilities is educational. Never test attacks on contracts you do not own, and do not publish working exploits for live contracts.

Check yourself

1. Why update state before sending funds?
2. What is a missing signer check on Solana?
3. What did Solidity 0.8 change about integers?

CONTENT HOOKS: POST ONE OF THESE

→ Reentrancy explained with a bank analogy
→ 5 questions I ask before trusting any smart contract
→ Solana security: the account checks beginners forget

CHAPTER 20

MEV, Sandwiches and Priority Fees

LESSON GOALS

- Explain how a sandwich attack works
- Reduce your exposure as a user
- Understand priority fees and bundles on Solana

MEV (maximal extractable value) is value that can be captured by choosing, ordering or inserting transactions. Most of the time it is invisible. A **sandwich attack** is the form that directly hurts ordinary traders.

How a sandwich works

1. You submit a large buy with generous slippage tolerance.
2. A bot sees it pending and buys the same token just before you, pushing the price up.
3. Your trade fills at the worse price, pushing the price up further.
4. The bot sells right after you and keeps the difference. You paid extra.

Reducing your exposure

- Keep slippage tolerance as tight as the trade can tolerate.
- Avoid large trades in thin pools.
- Use private or protected transaction routes where your wallet or tools offer them.

Priority fees and bundles on Solana

Solana lets you pay an extra **priority fee** to be scheduled sooner during busy periods. Some searchers use **bundles** (for example through Jito) to submit groups of transactions that execute together or not at all, often with a tip. Details evolve, so check current docs before you teach specifics.

CAUTION

Do not present MEV as an easy way to profit. It is highly competitive and technical, and most people who try it lose time and money.

Check yourself

1. What does a sandwich bot do?
2. What setting most affects sandwich risk?
3. What is a priority fee for?

CONTENT HOOKS: POST ONE OF THESE

→ How you get sandwiched in a trade (animated explainer)
→ Slippage settings: the one number that protects you
→ What Jito bundles are, in plain English

CHAPTER 21

Real-Time Data: Subscriptions and Watchers

LESSON GOALS

- Subscribe to on-chain events with websockets
- Build a devnet new-token watcher
- Handle rate limits and reconnects

Instead of asking the chain repeatedly, you can subscribe and be told when something happens. This is how dashboards, alerts and trackers stay live.

```
import { Connection, PublicKey, clusterApiUrl } from "@solana/web3.js";

const connection = new Connection(clusterApiUrl("devnet"), "confirmed");
const TOKEN_PROGRAM =
  new PublicKey("TokenkegQfeZyiNwAJbNbGKPFXCWuBvf9Ss623VQ5DA");

connection.onLogs(
  TOKEN_PROGRAM,
  ({ signature, logs }) => {
    if (logs.some((l) => l.includes("InitializeMint"))) {
      console.log("New mint transaction:", signature);
    }
  },
  "confirmed"
);
```

Run it, then create a token from another terminal and watch the signature appear. From the signature you can fetch the transaction and read the new mint's details with the code from the earlier chapter.

Making it reliable

- Expect disconnects: add reconnect logic with backoff.
- Respect provider rate limits and cache repeated lookups.
- Process events through a small queue so a slow lookup does not block new ones.
- For production, consider provider webhooks or an indexing service instead of raw subscriptions.
- Log everything with timestamps so you can debug and show your work.

Check yourself

1. Why subscribe instead of polling?
2. What should a watcher do when the connection drops?
3. What is the Token Program's role here?

CONTENT HOOKS: POST ONE OF THESE

→ Build a live new-token watcher in 30 lines
→ Websockets vs polling for blockchain data
→ My watcher caught a token being created in real time (devnet)

CHAPTER 22

Bots and Automation, Safely

LESSON GOALS

- Design a bot architecture
- Use paper mode first
- Apply security and risk rules to automation

Stage	Job	Example
Listener	Receives events	onLogs subscription for new mints
Filter	Drops irrelevant events quickly	Only token-program mint instructions
Analyzer	Fetches details and applies rules	Authorities, top-holder share, age
Logger	Stores every decision	File or database with timestamps
Executor	Acts on a result	Paper mode: record a simulated action only

```
// Rules engine skeleton (paper mode only)
const rules = [
  (t) => t.mintAuthority === null,   // authority revoked
  (t) => t.freezeAuthority === null, // freeze revoked
  (t) => t.topHolderPct < 30,        // not highly concentrated
];

function evaluate(token) {
  const passed = rules.filter((rule) => rule(token)).length;
  return { passed, total: rules.length };
}

// For each new mint: fetch details -> evaluate -> log the result
```

Safety rules

- Stay in paper mode until a tool has been tested for a long time on test networks.
- Keep keys in environment variables or a secure vault, never in code.
- If a bot ever touches real funds: small hard spending limits, a kill switch and alerts.
- Remember most trading bots lose money, and that you can lose funds through bugs alone.
- Read the terms of any API or platform you automate against.

For freelance work, tools that monitor, alert and analyze are easy to explain and to sell. They do not need to control anyone's funds.

Check yourself

1. What is paper mode?
2. Why log every decision?
3. Name two safety controls for a bot near real funds.

CONTENT HOOKS: POST ONE OF THESE

→ The 5 parts of every on-chain bot
→ My rules engine scored 20 devnet tokens: results
→ Why I only build monitoring tools, not auto-traders

CHAPTER 23

Solana Programs with Anchor

LESSON GOALS

- Understand what an Anchor program is
- Read and write a minimal counter program
- Run the build and test workflow

Solana programs are usually written in Rust. **Anchor** is a framework that reduces boilerplate and adds account validation through constraints, which helps prevent the account-check bugs from the security chapter.

```
use anchor_lang::prelude::*;

declare_id!("11111111111111111111111111111111"); // replace with your program id

#[program]
pub mod counter {
    use super::*;

    pub fn initialize(ctx: Context<Initialize>) -> Result<()> {
        ctx.accounts.counter.count = 0;
        Ok(())
    }

    pub fn increment(ctx: Context<Increment>) -> Result<()> {
        ctx.accounts.counter.count += 1;
        Ok(())
    }
}

#[derive(Accounts)]
pub struct Initialize<'info> {
    #[account(init, payer = user, space = 8 + 8)]
    pub counter: Account<'info, Counter>,
    #[account(mut)]
    pub user: Signer<'info>,
    pub system_program: Program<'info, System>,
}

#[derive(Accounts)]
pub struct Increment<'info> {
    #[account(mut)]
    pub counter: Account<'info, Counter>,
}

#[account]
pub struct Counter {
    pub count: u64,
}
```

- **#[program]** holds the instructions users can call.
- **#[derive(Accounts)]** lists the accounts each instruction needs, with rules such as init, mut and Signer.
- **#[account]** defines the data stored in an account. The 8 extra bytes in space are Anchor's account discriminator.

```
anchor init counter
cd counter
anchor build
```

```
anchor keys sync     # sync the program id into your code and config
anchor test          # runs the tests on a local validator
# set cluster = "devnet" in Anchor.toml, then: anchor deploy
```

NOTE

Commands, versions and project layout change between Anchor releases. Follow the official Anchor docs for your installed version, and test on devnet only.

Check yourself

1. What does Anchor add on top of raw Rust programs?
2. What does the Signer type enforce?
3. Why test on a local validator before devnet?

CONTENT HOOKS: POST ONE OF THESE

→ My first Solana program: a counter in Rust
→ Anchor constraints: how they stop common bugs
→ From 'hello world' to deployed on devnet

PART 5

Content, Freelance and Ethics

Skills only pay when people can find them. This part turns what you learn into a steady content flow, a believable freelance offer and a reputation you can protect.

CHAPTER 24

The Content Engine

LESSON GOALS

- Plan content around four pillars
- Use proven hook and slide formulas
- Repurpose one idea across X, Instagram and TikTok

Four content pillars

Pillar	Purpose	Example
Learn	Explain one concept clearly	How a bonding curve prices tokens
Build	Show something you made	Token due-diligence dashboard, week 2
Mistakes	Be honest about what broke	My transaction failed: here is why
Proof	Show results and client work	Case study: dApp front-end for a client

A healthy mix is about half Learn, a quarter Build, and the rest Mistakes and Proof.

Hook formulas

- How [thing] actually works, in [N] slides
- [N] things I wish I knew before [doing X]
- I built [X] so you do not have to
- Most people get [X] wrong. Here is the fix
- I tried [X] on devnet. Here is what happened
- Stop doing [X]. Do this instead

The carousel formula

1. **Slide 1**: the hook in large text.
2. **Slide 2**: why it matters or the problem.
3. **Slides 3 to 5**: one idea per slide, with a small diagram or code snippet.
4. **Slide 6**: a one-line recap.
5. **Slide 7**: call to action: follow for the next lesson, and DM for dev work.

The thread formula (X)

1. Post 1: the hook and a promise of what the reader will learn.
2. Posts 2 to 6: one point each, with an image or code snippet.
3. Final post: recap, link to your project or repo, and a clear invitation to follow or DM.

Repurposing workflow

1. Record once: the five-second intro clip and a short screen recording.
2. Build the slides from the chapter's Key terms and Try it sections.
3. Post on X first. A few hours later post the carousel on Instagram and photo mode or a short on TikTok.
4. Reply to every comment in the first hour.

What to measure

Followers are a vanity number. Watch **saves, shares, replies, profile visits and DMs**. Double down on the topics that lead to DMs and project enquiries.

Check yourself

1. Which four pillars structure your content?
2. What goes on slide 1 of a carousel?
3. Which metrics best predict freelance leads?

CONTENT HOOKS: POST ONE OF THESE

→ My content system as a developer (4 pillars)
→ How I turn one lesson into 3 posts
→ The 6 hook formulas I use every week

CHAPTER 25

30-Day Content Calendar

LESSON GOALS

- Have a post idea ready for every day for a month
- Tie every post to a chapter so study and content are the same work

Use this as a starting point and swap days around as you go. Each row maps to a chapter, so when you study the chapter you already know what to post.

Day	Source	Format	Post idea
1	Intro	Carousel	Web dev learning blockchain in public
2	Ch 1	Carousel	A blockchain explained with the shared-spreadsheet analogy
3	Ch 1	Clip	Hash demo: change one letter
4	Ch 2	Carousel	5 wallet security rules I follow
5	Ch 2	Clip	My dev lab: separate browser profile and test wallets
6	Ch 3	Carousel	Ethereum vs Solana for developers
7	Recap	Thread	Week 1: what I learned and what broke
8	Ch 4	Carousel	What a mint authority is
9	Ch 4	Clip	Decimals explained in 30 seconds
10	Ch 5	Clip	My first devnet airdrop
11	Ch 6	Carousel	The 6 stages of a memecoin
12	Ch 7	Carousel	How an AMM prices a trade
13	Ch 7	Clip	Slippage in one number
14	Recap	Thread	Week 2 recap
15	Ch 8	Carousel	Bonding curves explained
16	Ch 8	Clip	Build a bonding curve in 15 lines
17	Ch 9	Carousel	8 on-chain signals I check
18	Ch 10	Carousel	7 ways rugs work (concept only)
19	Ch 10	Carousel	My 10-step due-diligence checklist
20	Ch 11	Clip	Reading a token on an explorer
21	Recap	Thread	Week 3 recap
22	Ch 12	Carousel	Mistakes that wipe out most traders
23	Ch 13	Clip	My Web3 dev setup
24	Ch 14	Clip	Create a token in 3 commands on devnet
25	Ch 14	Clip	Revoking mint authority: before and after
26	Ch 15	Clip	An authorities checker in 10 lines

Day	Source	Format	Post idea
27	Ch 16	Carousel	Owner powers in a token contract
28	Recap	Thread	Week 4 recap and what comes next
29	Ch 17	Clip	Dashboard build log
30	Freelance	Carousel	What I build for clients, with proof

CHAPTER 26

Turning Skills into Freelance Income

LESSON GOALS

- Package your services clearly
- Reach out and run a good discovery call
- Protect yourself with scope, payment terms and boundaries

What you can offer

Service	Who needs it	Typical deliverable
dApp front-ends	Web3 startups and communities	Next.js app with wallet connect and read features
Token and project sites	New projects	Fast landing page with clear, honest information
Analytics dashboards	Communities and researchers	Live dashboard of holders, activity, pools
Wallet and payment integrations	Product teams	Connect, sign and display flows for web or mobile
Monitoring and alert tools	Traders and teams	Watcher with notifications, no custody of funds
Mobile apps	Founders	React Native / Expo app with Web3 features

Package it

Offer three clear packages: **Starter** (landing page and wallet connect), **Standard** (a full dApp front-end with read integrations) and **Premium** (custom dashboard plus backend). Price from what similar freelancers charge and the value of the result, then raise your prices as your proof grows. Avoid hourly pricing when you can.

Outreach message template

```
Hi [Name], I saw you are building [project]. I'm a full-stack dev
(Next.js and React Native) and I build Web3 front-ends and dashboards.
I recently built [specific project] - [link].
If you need help with [specific thing], happy to share a quick plan.
No pressure either way.
```

Discovery questions

- What problem should this solve, and for whom?
- What does success look like in 30 days?
- What exists already (designs, contracts, APIs)?
- Who decides, and what is the deadline?
- What is the budget range?

Protect yourself

- Write down the scope, the deliverables, the number of revisions and the timeline.
- Take payment in milestones. Use an escrow service for new clients.
- Keep ownership and licence terms clear in writing.
- Decline work that looks like a scam: fake-volume bots, drainer or phishing pages, tokens built to trap buyers, or anything you would be embarrassed to put in your portfolio.
- Never take custody of a client's private keys or funds.

Check yourself

1. Why take payment in milestones?
2. What kind of jobs should you decline?
3. What proof helps most in outreach?

CONTENT HOOKS: POST ONE OF THESE

→ What I build for Web3 founders (6 services)
→ How I scope a dApp project before writing code
→ Case study: a client project from brief to launch

CHAPTER 27

Ethics, Compliance and Reputation

LESSON GOALS

- Create content you can stand behind in 5 years
- Handle disclosure, risk and corrections

- **No shilling.** Do not promote tokens, and never promise returns or say a token will rise.
- **Disclose.** Say when you hold a token, use a referral link or are paid. Put it in the post, not buried.
- **Label your environment.** Show clearly when something is devnet or Sepolia.
- **Educate about risk.** Show how rugs and scams work so that people can protect themselves, not so that they can copy them.
- **Be careful about people.** On-chain data is public, but do not accuse named individuals without strong evidence, and do not publish private details.
- **Correct mistakes in public.** Pin the correction. It builds more trust than the error loses.
- **Check your local rules** on financial promotions, taxes and crypto services, especially before you earn from content or sell a paid group.
- **Protect yourself.** Use two-factor authentication, a hardware key where possible, and keep personal addresses and details private.

NOTE

A reputation as the developer who teaches honestly and ships real tools is rarer than a reputation for hype, and it is the one clients pay for.

Check yourself

1. What three things should you disclose?
2. What should you do after publishing a mistake?

CONTENT HOOKS: POST ONE OF THESE

→ Why I will never call a token on this account

→ How I handle sponsorships and disclosures

Appendices

A glossary, a tools list, a 12-week roadmap and the quiz answer key.

Appendix A: Glossary

Term	Meaning
AMM	Automated market maker: a contract that prices swaps from its reserves
Airdrop	Free tokens sent to wallets; on devnet, free test SOL
ATA	Associated token account: the standard token account for a wallet and a mint
Bonding curve	A rule that sets price from supply
Bundle	Many wallets acting together, often in the same block; or a set of transactions executed together
Burn	Permanently destroying tokens
DEX	Decentralized exchange
Devnet	Solana's developer test network
ERC-20	The standard token interface on Ethereum-style chains
FDV	Fully diluted valuation: price times total supply
Freeze authority	Account allowed to freeze token accounts (Solana)
Gas	The fee unit on Ethereum
Graduation	Moving a launchpad token's liquidity into a regular pool
Honeypot	A token that can be bought but not (easily) sold
Lamport	The smallest unit of SOL; 1 SOL = 1,000,000,000 lamports
Liquidity	The funds in a pool available to trade against
LP token	Receipt token representing a share of a liquidity pool
Mainnet	The real network where assets have value
MEV	Value extracted by ordering or inserting transactions
Mint	The account defining a token on Solana; also the act of creating supply
Mint authority	Account allowed to create more supply
Node	A computer that stores and verifies the chain
Price impact	How much your own trade moves the price
Priority fee	Extra Solana fee to be scheduled sooner
Private key	Secret that authorizes actions for an address
Rug pull	Controllers of a token or its liquidity taking value from holders
RPC	The endpoint your code uses to talk to a blockchain node
Sepolia	Ethereum's developer test network
Slippage	Difference between expected and received price

Term	Meaning
SPL token	A token created with Solana's Token Program
Sandwich attack	A bot buys before and sells after your trade to profit from your price impact
TVL	Total value locked in a protocol or pool
Wallet	App that holds your keys and signs transactions

Appendix B: Tools and Resources

Category	Tools
Wallets	Phantom, Solflare (Solana); MetaMask (EVM)
Explorers	Solscan, Solana Explorer, Etherscan
Analytics	DexScreener, Birdeye
RPC providers	Helius, Alchemy, QuickNode
Solana dev	Solana CLI, spl-token CLI, Anchor, @solana/web3.js
EVM dev	Remix, Foundry, Hardhat, OpenZeppelin
Web	Next.js, Vercel, Tailwind CSS
Design for content	Canva or Figma, one reusable dark template
Planning	A spreadsheet or Notion board for the content calendar
Docs to bookmark	Solana developer docs, Anchor docs, Solidity docs, OpenZeppelin docs

Tool names are examples, not endorsements. Check that a tool is current and reputable before you rely on it or recommend it.

Appendix C: 12-Week Roadmap

Week	Chapters	Deliverable
1	Ch 1 to 2	Mini blockchain script; dev browser profile and test wallets
2	Ch 3 to 5	Devnet airdrop; read three transactions on an explorer
3	Ch 6 to 7	AMM calculator script
4	Ch 8 to 9	Bonding-curve script; holder analysis of three tokens (study only)
5	Ch 10 to 12	Due-diligence checklist; paper-trading journal
6	Ch 13 to 14	CLI token created, minted, authority revoked
7	Ch 15	JavaScript authorities and holders checker
8	Ch 16	ERC-20 on Sepolia; contract-reading practice
9	Ch 17	Dashboard v1: API route and UI
10	Ch 17	Dashboard deployed with a written case study
11	Ch 19 to 21	Security review of your own contract; new-mint watcher
12	Ch 22 to 23	Rules engine in paper mode; Anchor counter on devnet

Appendix D: Quiz Answer Key

Chapter 1: What Is a Blockchain?

1. Block 1's hash changes, so block 2's stored previous-hash no longer matches. The chain is visibly broken from that point on.

2. Proof of Work: Bitcoin. Proof of Stake: Ethereum or Solana.

3. A node stores and checks the chain. A validator is a node that also helps produce or confirm blocks, usually by staking tokens.

Chapter 2: Wallets, Keys and Staying Safe

1. Only the public address.

2. A mistake, a malicious script or a bad signature in a dev environment should never be able to touch real funds.

3. It proves, with your private key, that you authorize that exact action.

Chapter 3: Transactions, Fees and Accounts

1. 1,000,000,000.

2. On Solana in separate data accounts owned by the program. On Ethereum inside the contract's own storage.

3. Yes.

Chapter 4: Tokens: Supply, Decimals and Authorities

1. The mint address (or contract address on EVM chains).

2. 5.0.

3. The holder of the authority can create unlimited new tokens and sell them, diluting everyone else.

Chapter 5: Testnets and Devnets: Your Free Lab

1. Devnet.

2. They have no value and can disappear on a reset. Offers to buy or sell them are usually scams.

3. The solana airdrop command or web faucet for devnet, and a Sepolia faucet for Ethereum testing.

Chapter 6: What Is a Memecoin?

1. Attention and the liquidity behind it, not cash flow or a product.

2. Creating a token is cheap and permissionless, and launchpads make it a few clicks.

3. It is useful, credible, avoids shilling, and shows real technical skill.

Chapter 7: Liquidity Pools and AMMs

1. The pool's token reserve falls and its SOL reserve rises, so the price per token goes up.

2. It removes a large share of one reserve, and the curve gets steep.

3. Only the liquidity in the pool is available to buyers, and it is usually far smaller than market cap.

Chapter 8: Bonding Curves and Launchpads

1. Price is a function of supply, so every token sold pushes the next price higher.

2. Moving the curve's remaining tokens and raised funds into a regular liquidity pool once a target is met.

3. A number used in the pricing maths that is not held as real funds, used to set the starting price.

Chapter 9: Supply, Holders and On-Chain Signals

1. Many separate wallets buying in the same block with funding that traces back to one source.
2. When a large part of the supply is locked or not yet released.
3. One actor can split funds across many wallets to look like a larger community.

Chapter 10: How Rugs Work and How to Spot Them

1. A token whose contract logic lets people buy but blocks or heavily taxes selling.
2. Insider dumps, bundles and thin liquidity can hurt buyers without any malicious code.
3. Mint/freeze authority, top-holder share, and whether liquidity is locked or burned, all visible on an explorer or analytics site.

Chapter 11: Reading On-Chain Data and Tools

1. A block explorer's holders view.
2. Scores can miss new patterns and give false alarms, so you must verify the underlying data.
3. It lets your audience reproduce your work and keeps you honest.

Chapter 12: Risk Management and Trader Psychology

1. Seeing only the winners' posts and ignoring the many who lost money.
2. Your reason for entering and the exit rule.
3. Any sponsorship, referral link or holding of that token.

Chapter 13: Your Developer Setup

1. It is a private key. Committing it to a public repo exposes it to everyone.
2. solana airdrop 2 (or the web faucet).
3. So your API key is not visible in the browser.

Chapter 14: Create Your First Token on Devnet

1. Who can create new supply.
2. No. Revoking is permanent.
3. Token balances are stored in token accounts, one per wallet per mint.

Chapter 15: Reading Chain Data with JavaScript

1. null.
2. The largest token accounts for a mint, which you can turn into a share of supply.
3. It can belong to a liquidity pool or a program.

Chapter 16: Your First EVM Token on Sepolia

1. Restricts a function so that only the contract's owner can call it.
2. Permanently removes the owner, so owner-only functions can never be called again.
3. It is widely reviewed and avoids easy mistakes.

Chapter 17: Flagship Project: Token Due-Diligence Dashboard

1. To protect the API key and control rate limits.
2. It keeps the project safe, legal and educational, with no trading advice.
3. A working demo, public code, and a short case study explaining the problem and result.

Chapter 18: More Build Projects

1. An AMM swap simulator or a bonding-curve visualizer.
2. A deployed demo, a clean README and a short write-up.

Chapter 19: Smart Contract Security Basics

1. So a re-entering attacker sees the updated balance and cannot withdraw twice.
2. A program that does not verify the required account actually signed the transaction.
3. Overflow and underflow now revert by default.

Chapter 20: MEV, Sandwiches and Priority Fees

1. Buys before your trade and sells after it, profiting from the price you moved.
2. Your slippage tolerance.
3. To get your transaction scheduled sooner when the network is busy.

Chapter 21: Real-Time Data: Subscriptions and Watchers

1. It is faster and uses fewer requests.
2. Reconnect with backoff and catch up on anything missed.
3. It handles token instructions, so its logs show new mints and transfers.

Chapter 22: Bots and Automation, Safely

1. Running the logic but only recording simulated actions.
2. It lets you debug, audit and learn from the bot's behaviour.
3. A hard spending limit and a kill switch.

Chapter 23: Solana Programs with Anchor

1. Less boilerplate and built-in account validation through constraints.
2. That the account actually signed the transaction.
3. It is faster and free of faucet and rate-limit problems.

Chapter 24: The Content Engine

1. Learn, Build, Mistakes and Proof.
2. A single large hook.
3. Saves, shares, replies, profile visits and DMs.

Chapter 26: Turning Skills into Freelance Income

1. It limits risk for both sides and keeps the project moving.
2. Anything built to deceive or harm users, such as drainer pages, fake-volume bots or trap tokens.
3. A relevant project you built, with a link.

Chapter 27: Ethics, Compliance and Reputation

1. Token holdings, referral links and sponsorships.
2. Correct it publicly and clearly.